

# An Object-Oriented Model Compression Planner for Resource-Constrained Binary Classification

202311294 Park Seonggwon

## Abstract

This project implements an object-oriented model compression planner for a simplified binary classification model. The goal of the system is to select compression actions that reduce memory usage while preserving the original prediction of the model. The proposed system represents model layers, compression actions, constraints, and planning logic as independent classes. Pruning and quantization are implemented as subclasses of a common `CompressionAction` interface, and prediction preservation is implemented as a reusable constraint object. Experimental results show that the planner can reduce the estimated model memory from 104.00 bytes to 10.50 bytes while preserving the original predictions on all validation samples. More importantly, the object-oriented structure improves modularity and extensibility because new compression actions or constraints can be added without rewriting the core planner.

## 1 Introduction

Deep learning models are increasingly deployed in resource-constrained environments such as mobile devices, embedded systems, and edge devices. However, neural networks often require a large amount of memory to store their parameters. When the available memory is limited, model compression techniques such as pruning and quantization become important for practical deployment.

From a programming perspective, implementing multiple compression methods in a single procedural function can quickly become difficult to maintain. For example, if pruning, 8-bit quantization, 4-bit quantization, and future compression methods are all handled by one function with many `if/elif` branches, the function becomes highly coupled. Adding a new method may require modifying stable existing code, which increases the risk of bugs.

In Assignment 1, inheritance was used to separate different compression layer behaviors such as a basic layer, a pruned layer, and a quantized layer. This project extends that idea from a simple layer-level example to a small object-oriented compression planning system. The system focuses on a simplified binary classifier and selects compression actions under two practical considerations: memory reduction and prediction preservation.

The objective of this project is not to propose a new model compression algorithm. Instead, the goal is to design and implement an extensible object-oriented software system for compression experiments. The main contributions are as follows. First, this project proposes a modular OOP architecture for a small model compression planning system. Second, it implements a polymorphic compression action interface that supports no compression, pruning, and quantization. Third, it implements a constraint-based evaluation mechanism that checks whether compressed models preserve the original binary classification predictions.

## 2 Related Work

### 2.1 Model Compression

Model compression aims to reduce the memory and computational cost of neural networks. Two representative techniques are pruning and quantization. Pruning removes or ignores less important parameters, often by setting small-magnitude weights to zero. Quantization reduces the numerical precision of parameters, for example from 32-bit floating point values to lower-bit representations.

Han et al. proposed the Deep Compression pipeline, which combines pruning, trained quantization, and Huffman coding to reduce the storage cost of neural networks. This shows that practical compression systems often involve multiple techniques rather than a single isolated method. Therefore, a software system for compression should be easy to extend when new techniques are added.

### 2.2 Object-Oriented Design in Machine Learning Systems

Modern machine learning frameworks commonly use object-oriented design. For example, neural network layers or modules are often represented as objects with common interfaces. This allows different components to be

composed flexibly while hiding internal implementation details. In this project, a similar idea is applied to model compression. Each compression method is represented as an object that implements the same interface.

## 2.3 Compression Planning

A compression technique defines how to modify a model, whereas a compression planner decides which technique should be applied. In real deployment scenarios, the best compression choice may depend on constraints such as memory budget, latency, accuracy, and hardware support. This project implements a simplified planner that searches for layer-wise compression actions while checking whether the compressed model preserves the original prediction.

# 3 Proposed OOP System

## 3.1 System Overview

The proposed system consists of four main components: model representation, compression actions, constraints, and planner logic. The model is represented as a `BinaryClassifier`, which is composed of multiple `LayerBlock` objects. Compression methods are represented as subclasses of `CompressionAction`. Constraints are represented as subclasses of `CompressionConstraint`. Finally, the `GreedyCompressionPlanner` evaluates candidate actions and selects a compression plan.

**Figure 1. Overall architecture of the proposed system**

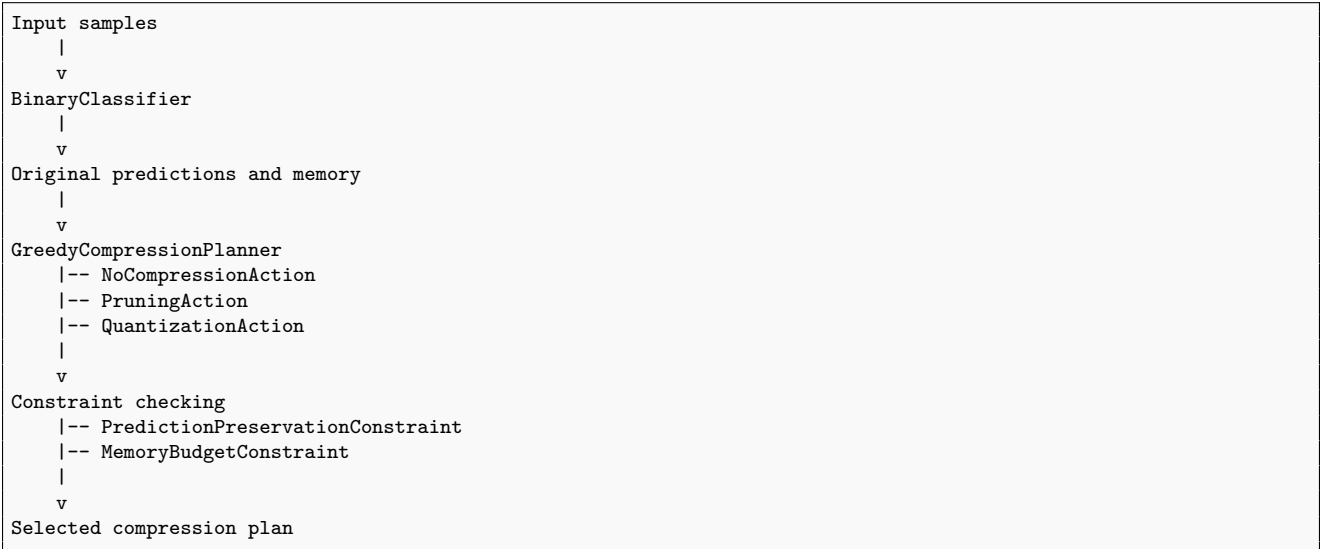


Figure 1 shows the overall workflow of the proposed system. The input samples are first passed through the original binary classifier. The planner then evaluates candidate compression actions and checks whether each candidate satisfies the prediction preservation constraint. Finally, the planner selects a compression plan that reduces memory while preserving the original predictions.

## 3.2 Class Design

The proposed system is designed using several classes, each with a separate responsibility. This separation is important because a compression system may need to support many different techniques and constraints.

## 3.3 OOP Principles Used in the System

Encapsulation is used in the `LayerBlock` class. The class stores layer-specific data such as weights, bias, activation type, bit-width, and storage format. Other classes do not need to manually calculate the memory or directly manage the internal layer representation. Instead, they can call methods such as `forward()` and `memory_bytes()`.

Abstraction is used in the `CompressionAction` class. This abstract parent class defines that every compression action must implement an `apply()` method and a `name()` method. The planner can rely on this abstract interface without knowing the detailed implementation of each compression method.

Inheritance is used when concrete compression actions such as `PruningAction` and `QuantizationAction` inherit from `CompressionAction`. Similarly, concrete constraints inherit from `CompressionConstraint`. This allows the system to share a common interface while supporting different internal behaviors.

Table 1: Class design of the proposed OOP system.

| Class                                   | Responsibility                                                             | OOP Concept   |
|-----------------------------------------|----------------------------------------------------------------------------|---------------|
| <b>LayerBlock</b>                       | Stores layer weights, bias, activation, bit-width, and memory estimation.  | Encapsulation |
| <b>BinaryClassifier</b>                 | Composes multiple layer objects and produces binary classification logits. | Composition   |
| <b>CompressionAction</b>                | Defines the common interface for all compression methods.                  | Abstraction   |
| <b>NoCompressionAction</b>              | Keeps a layer unchanged.                                                   | Inheritance   |
| <b>PruningAction</b>                    | Applies magnitude-based pruning to a layer.                                | Polymorphism  |
| <b>QuantizationAction</b>               | Applies simplified low-bit quantization to a layer.                        | Polymorphism  |
| <b>CompressionConstraint</b>            | Defines the common interface for constraints.                              | Abstraction   |
| <b>PredictionPreservationConstraint</b> | Checks whether predictions are preserved after compression.                | Inheritance   |
| <b>MemoryBudgetConstraint</b>           | Checks whether the compressed model satisfies the memory budget.           | Inheritance   |
| <b>GreedyCompressionPlanner</b>         | Searches layer-wise compression actions and selects a plan.                | Composition   |

Polymorphism is one of the most important concepts in this project. The planner calls `action.apply(layer)` for every compression action. The same method call produces different behaviors depending on the actual subclass. For example, **PruningAction** replaces small weights with zero, while **QuantizationAction** reduces numerical precision. Therefore, the planner does not need separate conditional branches for each compression type.

Composition is used because the **BinaryClassifier** is composed of **LayerBlock** objects. The planner is also composed of compression action objects and constraint objects. This composition-based structure makes the overall system flexible and modular.

### 3.4 Planning Algorithm

The planner uses a simple layer-wise greedy search. For each layer, it tries all candidate compression actions and checks whether the candidate model preserves the original predictions on the validation samples. Among accepted candidates, it selects the one with the smallest memory usage. The process is repeated for every layer.

#### Figure 2. Simplified greedy planning algorithm

```

for each layer in model:
    best_action = NoCompression
    for each action in candidate_actions:
        candidate_model = apply action to current layer
        if candidate_model preserves original predictions:
            if candidate_model uses less memory:
                best_action = action
    update current model using best_action

```

This algorithm is not guaranteed to find the globally optimal compression plan. However, it is simple, understandable, and suitable for demonstrating how object-oriented design can organize compression planning logic.

## 4 Implementation and Results

### 4.1 Implementation Environment

The system was implemented in Python using only the standard library. No external deep learning framework was required because the purpose of this project is to demonstrate object-oriented design rather than train a real neural network. The toy model consists of one hidden layer and one output layer. The hidden layer uses ReLU activation, and the output layer produces two logits for binary classification.

### 4.2 Core Implementation

The following code shows the common parent class for compression actions and two concrete subclasses.

```

class CompressionAction(ABC):
    @abstractmethod
    def apply(self, layer):
        pass

    @abstractmethod
    def name(self):
        pass

class PruningAction(CompressionAction):
    def __init__(self, threshold):
        self.threshold = threshold

    def apply(self, layer):
        new_layer = layer.clone()
        new_layer.weights = [
            [0.0 if abs(w) <= self.threshold else w for w in row]
            for row in layer.weights
        ]
        new_layer.sparse = True
        return new_layer

    def name(self):
        return f"Pruning(threshold={self.threshold})"

class QuantizationAction(CompressionAction):
    def __init__(self, bit_width):
        self.bit_width = bit_width

    def apply(self, layer):
        new_layer = layer.clone()
        scale = (2 ** (self.bit_width - 1)) - 1
        new_layer.weights = [
            [round(w * scale) / scale for w in row]
            for row in layer.weights
        ]
        new_layer.bias = [round(b * scale) / scale for b in layer.bias]
        new_layer.bit_width = self.bit_width
        return new_layer

    def name(self):
        return f"{self.bit_width}-bit Quantization"

```

This implementation shows the benefit of polymorphism. The planner does not need to check whether an action is pruning or quantization. It simply calls `apply()`.

### 4.3 Experimental Setting

The original model has two layers: `HiddenLayer` and `OutputLayer`. The memory is estimated using a simplified memory model. Dense parameters are stored according to their bit-width. Sparse layers store only nonzero weights with a small index overhead. This memory model is not hardware-accurate, but it is useful for comparing candidate compression actions in this project.

The validation dataset contains four input samples. The original model predictions are used as reference predictions. A compression candidate is accepted only when it preserves all reference predictions.

Table 2: Original model predictions.

| Sample | Logits            | Prediction |
|--------|-------------------|------------|
| 0      | [0.8900, -0.6400] | Class 0    |
| 1      | [-0.5800, 0.8200] | Class 1    |
| 2      | [0.1860, 0.2060]  | Class 1    |
| 3      | [1.3760, -0.8015] | Class 0    |

Sample 2 has a small logit margin, so it is useful for testing whether aggressive compression changes the model prediction.

## 4.4 Execution Result

After implementing the proposed classes, the program was executed on the toy binary classification model. The planner evaluates multiple compression actions, including no compression, pruning, and quantization. For each candidate action, the program reports the estimated memory usage, memory reduction ratio, whether the original prediction is preserved, and whether the candidate is accepted by the planner. The complete terminal output is shown in Figure 1.

```
Original model architecture
HiddenLayer: activation=relu, bit_width=32, storage=dense, memory=64.00 bytes
OutputLayer: activation=identity, bit_width=32, storage=dense, memory=40.00 bytes
Total memory: 104.00 bytes

Original predictions
sample 0: logits=[0.8899999999999999, -0.64], prediction=class 0
sample 1: logits=[-0.5799999999999998, 0.8199999999999998], prediction=class 1
sample 2: logits=[0.18599999999999994, 0.206], prediction=class 1
sample 3: logits=[1.3760000000000001, -0.8015000000000001], prediction=class 0

Candidate evaluation history


| Layer       | Action                 | Memory  | Reduction | Preserved | Accepted |
|-------------|------------------------|---------|-----------|-----------|----------|
| HiddenLayer | No Compression         | 104.00B | 0.00%     | True      | True     |
| HiddenLayer | Pruning(threshold=0.2) | 101.00B | 2.88%     | True      | True     |
| HiddenLayer | Pruning(threshold=0.4) | 81.00B  | 22.12%    | True      | True     |
| HiddenLayer | 8-bit Quantization     | 56.00B  | 46.15%    | True      | True     |
| HiddenLayer | 4-bit Quantization     | 48.00B  | 53.85%    | True      | True     |
| HiddenLayer | 2-bit Quantization     | 44.00B  | 57.69%    | False     | False    |
| OutputLayer | No Compression         | 48.00B  | 53.85%    | True      | True     |
| OutputLayer | Pruning(threshold=0.2) | 51.00B  | 50.96%    | False     | False    |
| OutputLayer | Pruning(threshold=0.4) | 41.00B  | 60.58%    | False     | False    |
| OutputLayer | 8-bit Quantization     | 18.00B  | 82.69%    | True      | True     |
| OutputLayer | 4-bit Quantization     | 13.00B  | 87.50%    | True      | True     |
| OutputLayer | 2-bit Quantization     | 10.50B  | 89.90%    | True      | True     |



Selected compression plan
HiddenLayer: 4-bit Quantization
OutputLayer: 2-bit Quantization

Final result
Original memory: 104.00 bytes
Final memory: 10.50 bytes
Memory reduction: 89.90%
Prediction preserved: True
Memory budget satisfied: True
```

Figure 1: Terminal execution result of the proposed OOP-based compression planner. The output shows the original model architecture, original predictions, candidate evaluation history, selected compression plan, and final memory reduction result.

As shown in Figure 1, the original model consists of two layers: **HiddenLayer** and **OutputLayer**. The original memory usage is 104.00 bytes. The original model predicts class 0 for samples 0 and 3, and class 1 for samples 1 and 2. These original predictions are used as the reference outputs for checking prediction preservation after compression.

## 4.5 Result Analysis

Table 3 summarizes the final result. The planner successfully reduces the estimated memory from 104.00 bytes to 10.50 bytes, corresponding to an 89.90% memory reduction. At the same time, the compressed model preserves the original predictions on all validation samples and satisfies the memory budget.

Table 3: Summary of the final compression result.

| Item                    | Value              |
|-------------------------|--------------------|
| Original memory         | 104.00 bytes       |
| Final memory            | 10.50 bytes        |
| Memory reduction        | 89.90%             |
| HiddenLayer action      | 4-bit Quantization |
| OutputLayer action      | 2-bit Quantization |
| Prediction preserved    | True               |
| Memory budget satisfied | True               |

The candidate evaluation history shows that the planner does not simply choose the most aggressive compression action. For the hidden layer, 2-bit quantization reduces the memory to 44.00 bytes, but it does not preserve the original predictions. Therefore, this candidate is rejected. Instead, the planner selects 4-bit quantization for the hidden layer because it reduces memory while preserving all original predictions.

For the output layer, 2-bit quantization is accepted because it preserves the original predictions after the hidden layer has already been compressed. The final selected compression plan is therefore heterogeneous: the hidden layer uses 4-bit quantization, while the output layer uses 2-bit quantization. This result shows that different layers can have different compression sensitivity.

From an object-oriented programming perspective, the important point is that the planner can evaluate different compression actions through the same interface. The planner calls `action.apply(layer)` regardless of whether the action is pruning, 8-bit quantization, 4-bit quantization, or 2-bit quantization. This demonstrates polymorphism. If a new compression method is added later, the core planner does not need to be rewritten. A new subclass of `CompressionAction` can be implemented and added to the action list.

## 5 Limitations

This project has several limitations. First, the model is a simplified binary classifier rather than a real neural network trained on a real dataset. Therefore, the numerical result should be interpreted as a software design demonstration rather than a real compression benchmark.

Second, the quantization implementation is simplified. It simulates lower numerical precision by rounding values, but it does not implement real integer arithmetic or hardware-aware quantization.

Third, the memory estimator is also simplified. Real deployment memory depends on many factors, including tensor layout, runtime buffers, sparse format overhead, and hardware support.

Fourth, the planner uses a greedy layer-wise strategy. This means it does not guarantee a globally optimal compression plan. Compression actions can interact across layers, and a locally good choice may not always be globally optimal.

Finally, prediction preservation is checked only on a small validation set. A real system should evaluate accuracy, latency, and memory on a larger validation dataset.

## 6 Conclusion

This project implemented an object-oriented model compression planner for a simplified binary classification model. The system separates model layers, compression actions, constraints, and planning logic into independent classes. This design demonstrates key OOP principles such as encapsulation, abstraction, inheritance, polymorphism, and composition.

The experimental result showed that the planner reduced the estimated model memory from 104.00 bytes to 10.50 bytes while preserving the original predictions on all validation samples. More importantly, the project showed that object-oriented programming is useful not only for code reuse but also for designing extensible machine learning software systems. This is directly related to model compression research, where different compression methods and deployment constraints must be tested and combined in a maintainable way.

## References

- Han, S., Mao, H., and Dally, W. J. (2016). Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. International Conference on Learning Representations.
- Python Software Foundation. Classes. Python Documentation.
- PyTorch Contributors. `torch.nn.Module`. PyTorch Documentation.